

CLASE 1 · UNIDAD 1 · SEMANA 1

De vectores a un LLM

Fundamentos matemáticos del aprendizaje profundo

Inteligencia Artificial Generativa Avanzada y Sistemas Multi Agente

Prof. Francisco Traversaro — Doctor en Ingeniería (ITBA)

AlixPartners

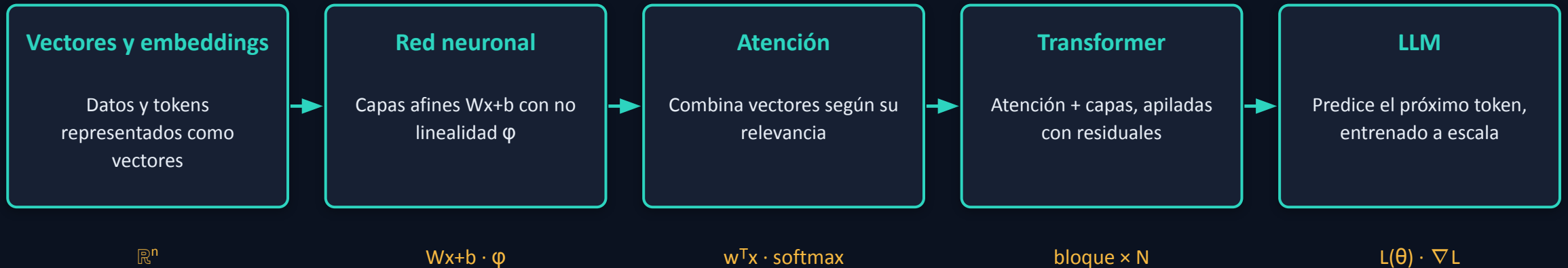
Docente · Licenciatura en Tecnología Digital, UTDT

[in/frantraversaro](#)



Cómo se llega a un LLM

El recorrido en bloques: de los vectores al Transformer. En la segunda parte definiremos los símbolos de cada bloque.



Los símbolos de esta unidad ($\mathbb{R}^n$, $w^T x$, $Wx+b$, ϕ , L , ∇) son las piezas de estos bloques.

La clave. todo LLM se apoya en estos ladrillos; entender los ladrillos es entender el modelo. Hoy construimos los cimientos.

Qué construimos: objetos y notación

Un mapa de objetos, no una cronología. Y los símbolos que vas a ver toda la clase.



último nodo (AD) = el algoritmo que hace todo esto entrenable

Convenciones de notación

$$x \in \mathbb{R}^n$$

— vectores

$$W \in \mathbb{R}^{m \times n}$$

— matrices / capas lineales

$$\theta \in \mathbb{R}^p$$

— parámetros a aprender

$$w^T x$$

— producto interno

$$\nabla$$

— gradiente

$$\phi$$

— función de activación

$$\eta$$

— tasa de aprendizaje

Hoja de ruta de la clase

Dos bloques de 95 min (~190 min) · teoría y práctica mezcladas.

1

Encuadre

~15 min

objetos y notación

2

Fundamentos

~80 min

de vectores a la optimización

3

Lab: autograd

~80 min

construimos un motor de diferenciación

4

Síntesis

~15 min

la cadena, completa

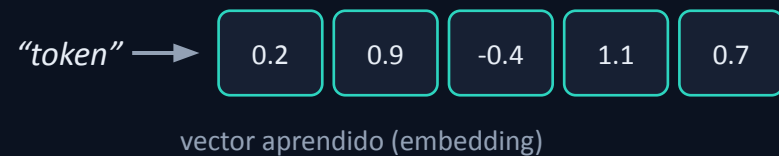
Cómo trabajamos. materia práctica: teoría y laboratorio juntos, y proyectos en grupo. Lo marcado como opcional (en láminas aparte) es profundidad extra para quien tenga más base — no bloquea a nadie.

Vectores y el espacio $\mathbb{R}^n$

$$\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$$

$$\|\mathbf{x}\| = \sqrt{x_1^2 + \dots + x_n^2}$$

Dos lecturas equivalentes: n-tupla de coordenadas (álgebra) y punto/dirección con norma (geometría). En deep learning todo dato — un token, una imagen — se codifica como vector en dimensión alta: representación distribuida.



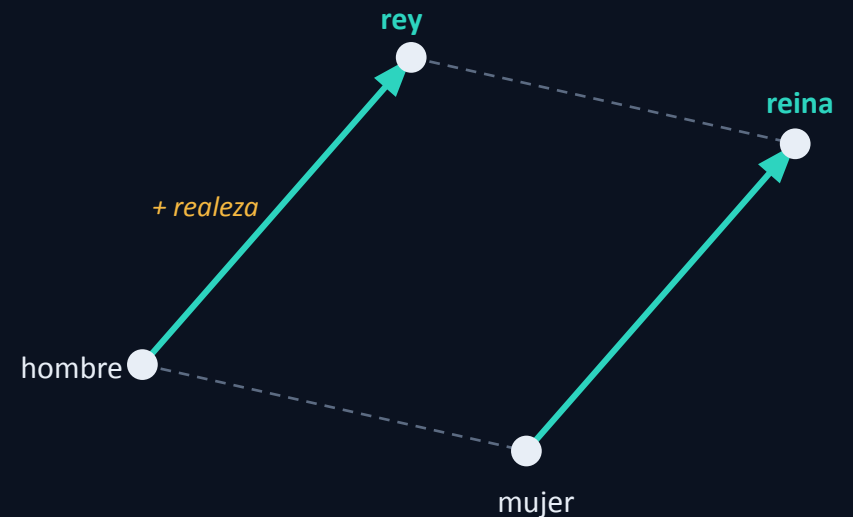
Una idea de embeddings

Si aprendemos un vector por palabra, la geometría del espacio termina capturando significado: palabras parecidas caen cerca y ciertas direcciones tienen sentido.

$$\text{vec}(\text{rey}) - \text{vec}(\text{hombre}) + \text{vec}(\text{mujer}) \approx \text{vec}(\text{reina})$$

La resta $\text{rey} - \text{hombre}$ aísla la idea de “realeza”; sumársela a mujer cae cerca de reina. La misma dirección sirve para muchos pares (país→capital, verbo→tiempo).

Lo vemos en profundidad más adelante; por ahora, sólo la intuición.



Espacios vectoriales y normas L_p

Espacio vectorial

Un conjunto V con suma y producto por escalar (sobre $\mathbb{R}$) que cumple: asociatividad y conmutatividad de $+$, vector nulo 0 , opuesto $-x$, las dos distributividades y $1 \cdot x = x$.

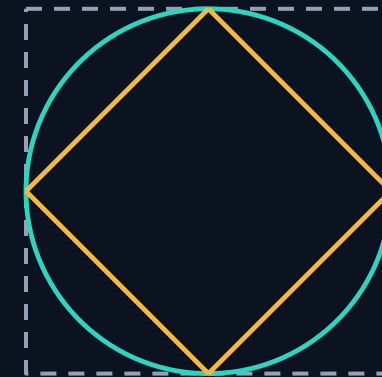
Norma L_p

$$\|x\|_p = \left(\sum_i |x_i|^p \right)^{1/p}$$

$p=1$ Manhattan $p=2$ euclídea $p=\infty$ máximo

En ML: $L2$ para distancias y regularización; $L1$ induce esparsidad (muchos ceros).

Bolas unidad $\|x\| = 1$



$L1 \cdot L2 \cdot L^\infty$

Producto interno: la operación central

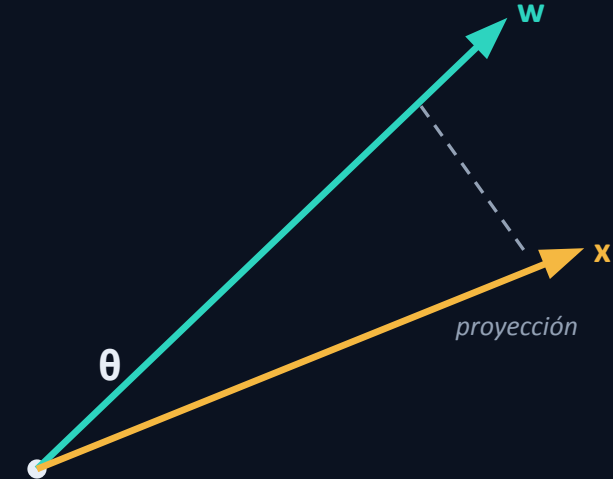
$$w^T x = \sum_i w_i x_i$$

$$w^T x = \|w\| \|x\| \cos \theta$$

$$\cos \theta = w^T x / (\|w\| \|x\|)$$

Mide alineación / proyección entre dos vectores; el signo y la magnitud dependen del ángulo θ . Es la base de la similitud coseno con que se comparan embeddings.

- **CLAVE** en atención, el peso de un token sobre otro es el producto interno de sus vectores query y key (escalado por $\sqrt{d}$, normalizado con softmax). Es esta misma operación.



Desigualdad de Cauchy–Schwarz

$$|w^T x| \leq \|w\| \|x\|$$

con igualdad si y sólo si w y x son colineales.

Consecuencia

$$-1 \leq \cos \theta \leq 1$$

Por eso la similitud coseno está bien definida y acotada — la base de comparar embeddings.

Idea de la prueba

$$\|x - tw\|^2 \geq 0 \quad \forall t \in \mathbb{R}$$

$$\|w\|^2 t^2 - 2(w^T x) t + \|x\|^2 \geq 0$$

Cuadrática en t siempre $\geq 0 \Rightarrow$ discriminante ≤ 0 :

$$(w^T x)^2 - \|w\|^2 \|x\|^2 \leq 0$$

$$\Rightarrow (w^T x)^2 \leq \|w\|^2 \|x\|^2$$

Tomando raíz se obtiene la desigualdad.

Aplicaciones lineales y matrices

$W \in \mathbb{R}^{m \times n}$ representa $\mathbb{R}^n \rightarrow \mathbb{R}^m$

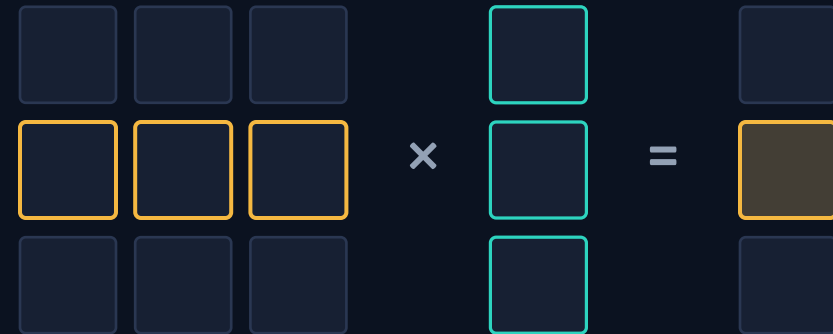
$$(Wx)_i = (\text{fila}_i)^T x$$

lote: $W X, X \in \mathbb{R}^{n \times B}$

Cada componente de la salida es un producto interno de una fila con la entrada: una capa lineal son m productos internos en paralelo.

Componer capas lineales = multiplicar matrices.

El producto matriz-matriz domina el costo del entrenamiento $\Rightarrow$ hardware paralelo (GPU).



$$(Wx)_i = (\text{fila}_i)^T x$$

fila i · vector = componente i

Rango, cambio de base y geometría

Rango

$\text{rango}(W)$ = nº de direcciones independientes que W puede producir (dimensión del espacio columna). Rango bajo = cuello de botella / compresión.

Valores singulares (SVD)

$$W = U \Sigma V^T$$

Toda aplicación lineal es rotación · escala · rotación; Σ dice cuánto estira cada dirección.

Relevante para atención de rango bajo y adaptadores como LoRA.



Unidad afín + no linealidad

$$a = \varphi(w^T x + b) \quad (\text{unidad})$$

$$h = \varphi(Wx + b) \quad (\text{capa; } \varphi \text{ elementwise})$$

Transformación afín (reconoce el producto interno $w^T x$) seguida de una no linealidad φ .

Proposición. La composición de aplicaciones afines es afín. Sin φ , apilar capas no agrega expresividad: la red colapsa a una sola capa lineal.



ReLU

 σ 

tanh

Composición afín y aproximación universal

Por qué hace falta ϕ

$$\begin{aligned}f_2(f_1(x)) &= A_2(A_1 x + b_1) + b_2 \\ &= (A_2 A_1) x + (A_2 b_1 + b_2)\end{aligned}$$

Sigue siendo afín: sin no linealidad, apilar capas no agrega expresividad.

Teorema de aproximación universal

Una red con una capa oculta y una no linealidad no polinómica aproxima cualquier función continua sobre un compacto, con suficiente ancho (Cybenko 1989; Hornik 1991).

Matiz: no dice cuántas neuronas ni si el entrenamiento la encuentra. La profundidad ayuda en la práctica.

aproximar una curva con tramos lineales



Aprendizaje supervisado como optimización

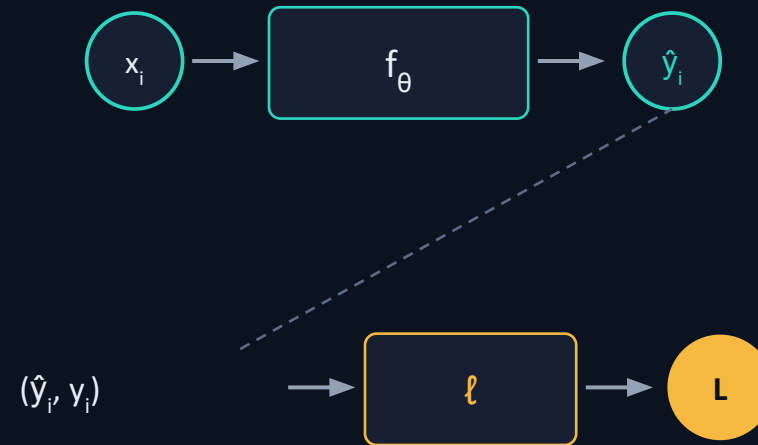
datos: $(x_1, y_1), \dots, (x_N, y_N)$

modelo f_θ , parámetros $\theta \in \mathbb{R}^p$

$$L(\theta) = (1/N) \sum_i \ell(f_\theta(x_i), y_i)$$

$\min_\theta L(\theta)$ (en general no convexo)

Entrenar = resolver un problema de optimización en $\mathbb{R}^p$. Sin garantía de óptimo global: buscamos buenos mínimos locales. Riesgo empírico $\neq$ riesgo esperado (generalización, Unidad 2).



p. ej. $\hat{y}_i = 0.8, y_i = 1 \Rightarrow \ell$ mide el error

Generalización: riesgo empírico vs esperado

$$R(\theta) = E_{(x,y) \sim D} [\ell(f_{\theta}(x), y)]$$

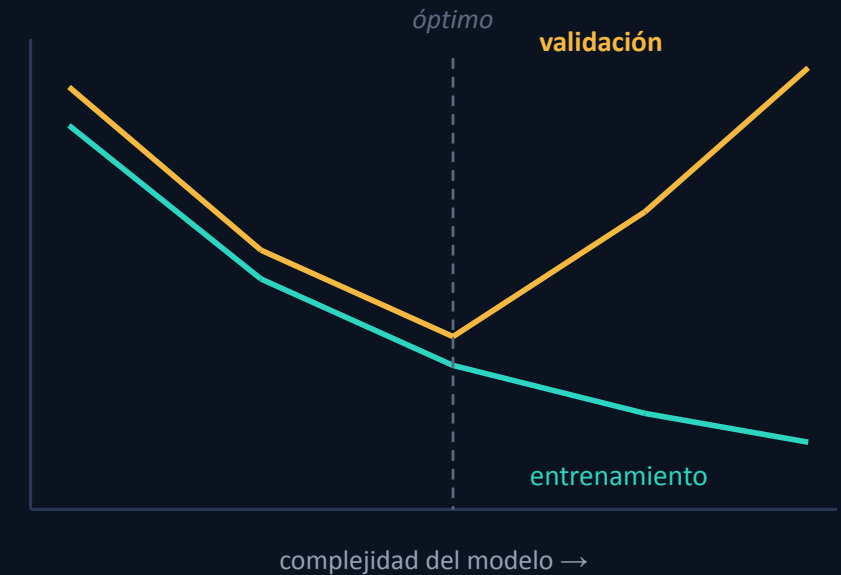
lo que querríamos minimizar (sobre datos futuros)

$$L(\theta) = (1/N) \sum_i \ell(f_{\theta}(x_i), y_i)$$

lo que efectivamente medimos (sobre el conjunto de entrenamiento)

Gap de generalización = $R - L$. Sobreajuste: L baja pero R alta (el modelo memoriza en lugar de aprender).

Herramientas (Unidad 2): partición train / val / test, regularización, early stopping.



Funciones de pérdida

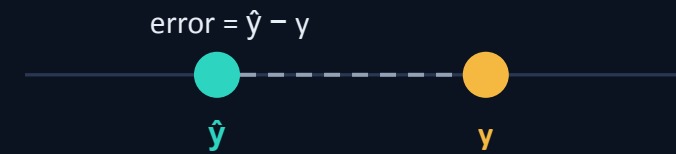
Regresión: $\ell(\hat{y}, y) = \frac{1}{2} \|\hat{y} - y\|^2$

Clasificación: $\hat{y} = \text{softmax}(z)$

$\ell = - \sum_i y_i \log \hat{y}_i$ (entropía cruzada)

softmax normaliza los logits z a una distribución de probabilidad; la entropía cruzada es la log-verosimilitud negativa de la clase correcta. La pérdida codifica el supuesto del problema: continuo $\Rightarrow$ MSE, categórico $\Rightarrow$ cross-entropy.

regresión



clasificación (softmax)



$\Sigma = 1$ · clase correcta resaltada

Pérdidas desde máxima verosimilitud

MSE ← modelo gaussiano

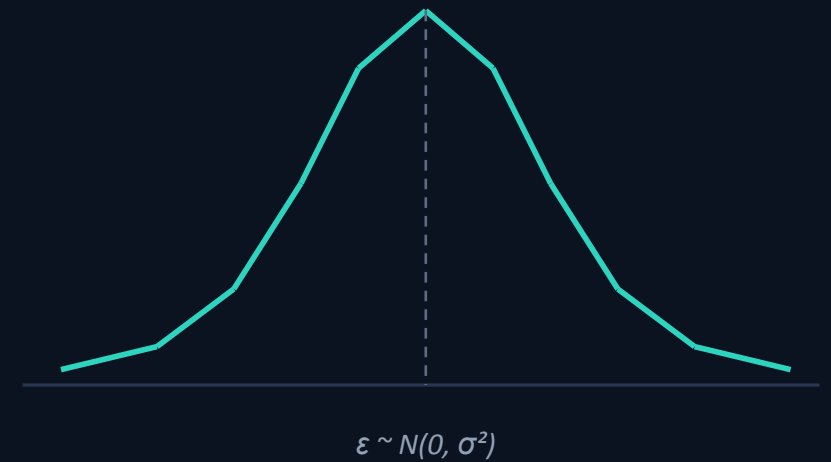
Si $y = f_{\theta}(x) + \varepsilon$ con $\varepsilon \sim N(0, \sigma^2)$, maximizar la verosimilitud equivale a minimizar $\sum \|y - \hat{y}\|^2$ — es el MSE.

Cross-entropy ← modelo categórico

Si $p_{\theta}(y|x) = \text{softmax}(z)$, la log-verosimilitud de la clase correcta es $\sum \log \hat{y}_i$; maximizarla = minimizar $-\sum y_i \log \hat{y}_i$ — la entropía cruzada.

La idea. elegir una pérdida es, en el fondo, elegir un modelo probabilístico de los datos.

verosimilitud gaussiana del error

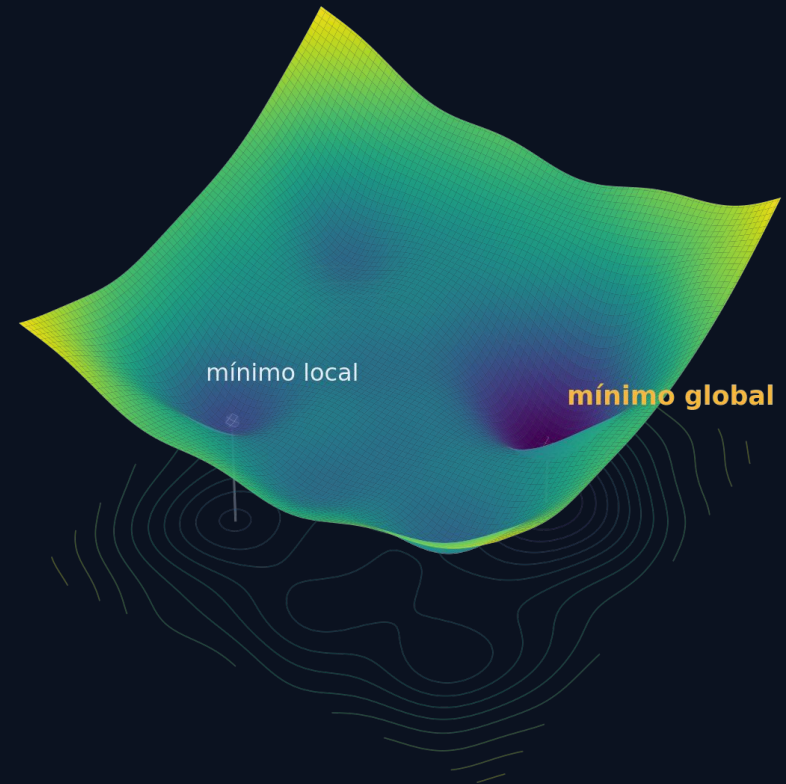


Minimizar $L(\theta)$: un paisaje complicado

Entrenar es buscar el θ que minimiza $L(\theta)$. Pero L no es una taza convexa: en una red real es una superficie de dimensión altísima, llena de valles, crestas y mesetas.

No podemos despejar el mínimo con una fórmula: hay millones de parámetros y muchísimos puntos donde $\nabla L = 0$ (mínimos locales, sillas). Encontrar el mínimo global exacto es inviable.

La idea. descendemos paso a paso siguiendo la pendiente, y nos conformamos con un buen mínimo local. Esa pendiente es el gradiente.



una pérdida no convexa: muchos mínimos locales, un solo mínimo global

Gradiente y descenso por gradiente

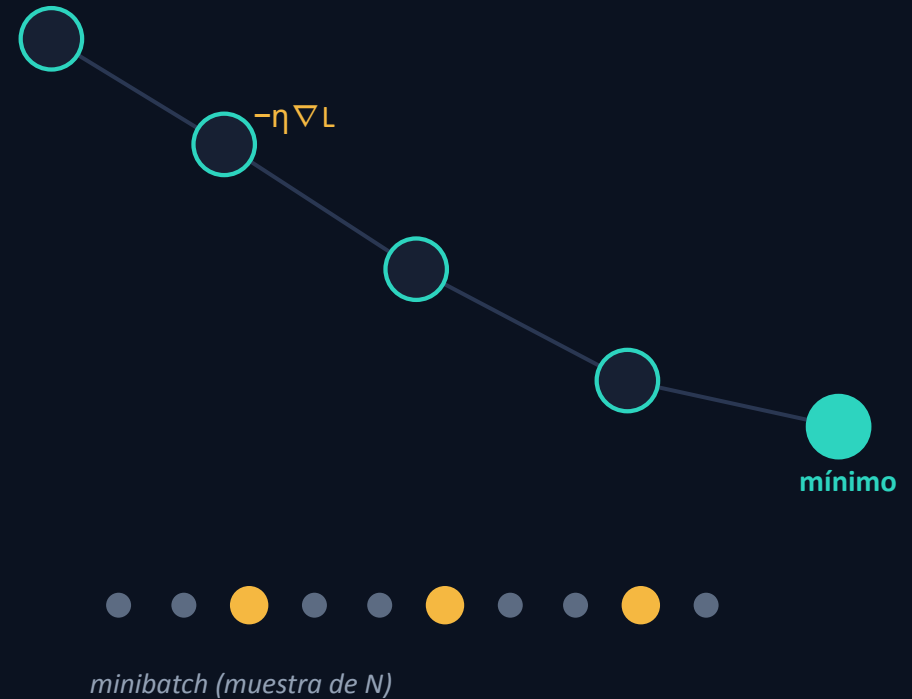
$$\nabla_{\theta} L \in \mathbb{R}^p \text{ (derivadas parciales)}$$

apunta a la dirección de máximo ascenso de L

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$$

Minimizamos $\Rightarrow$ nos movemos en $-\nabla$, escalado por la tasa η . η grande: diverge; η chico: converge lento.

SGD: ∇ estimado sobre un minibatch (insesgado)



Derivada direccional, momentum y Adam

Por qué ∇ es la dirección de máximo ascenso

$$D_u L(\theta) = \nabla L \cdot u$$

La tasa de cambio en la dirección unitaria u es máxima cuando u apunta como ∇L .

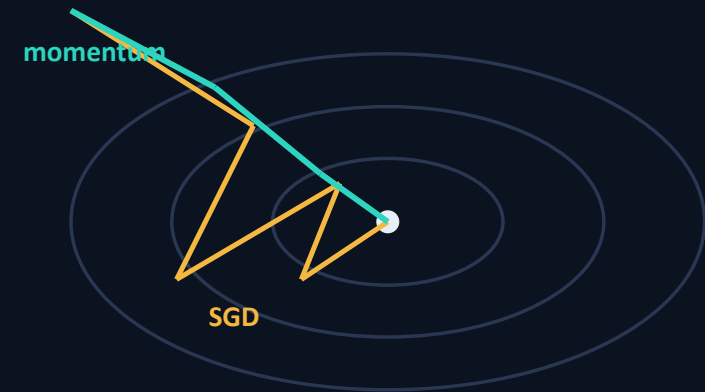
Mejoras del paso (Unidad 2)

Momentum: $v \leftarrow \beta v + \nabla L$, $\theta \leftarrow \theta - \eta v$

Acumula inercia: atraviesa mesetas y ravinas más rápido.

Adam: momentum + escala adaptativa por coordenada

Medias móviles de ∇ y ∇^2 ; el optimizador por defecto en la práctica.



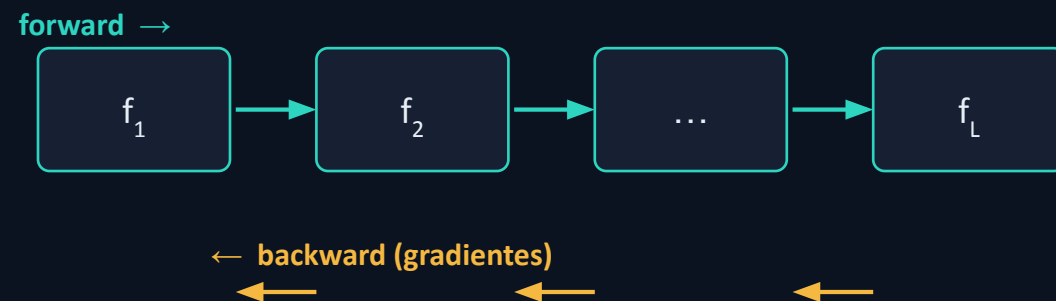
Regla de la cadena y modo reverso

$$f_{\theta} = f_L \circ \dots \circ f_1 \text{ (composición)}$$

$$\partial z / \partial x = (\partial z / \partial y)(\partial y / \partial x)$$

Backprop = evaluar esa cadena de la salida hacia la entrada, reutilizando resultados (producto de Jacobianos).

Por qué reverso. 1 escalar de salida (la pérdida) y p parámetros $\Rightarrow$ el modo reverso obtiene ∇ completo en UNA pasada hacia atrás, a costo $O(\text{forward})$. El modo directo costaría $O(p)$.



un escalar $\leftarrow$ muchos parámetros

Jacobianos y modos de diferenciación

Jacobiano

$$f: \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad J \in \mathbb{R}^{m \times n}$$

$$J_{ij} = \partial f_i / \partial x_j$$

La regla de la cadena vectorial es un producto de Jacobianos.

Dos modos de recorrerla

Forward: de entradas a salida — costo $\propto$ nº de entradas.

Reverse: de salida a entradas — costo $\propto$ nº de salidas.

forward-mode



una pasada por cada entrada

reverse-mode



una pasada por cada salida

Por qué reverse. 1 salida (la pérdida) y millones de parámetros $\Rightarrow$ una sola pasada hacia atrás da todos los gradientes. Eso es backprop.

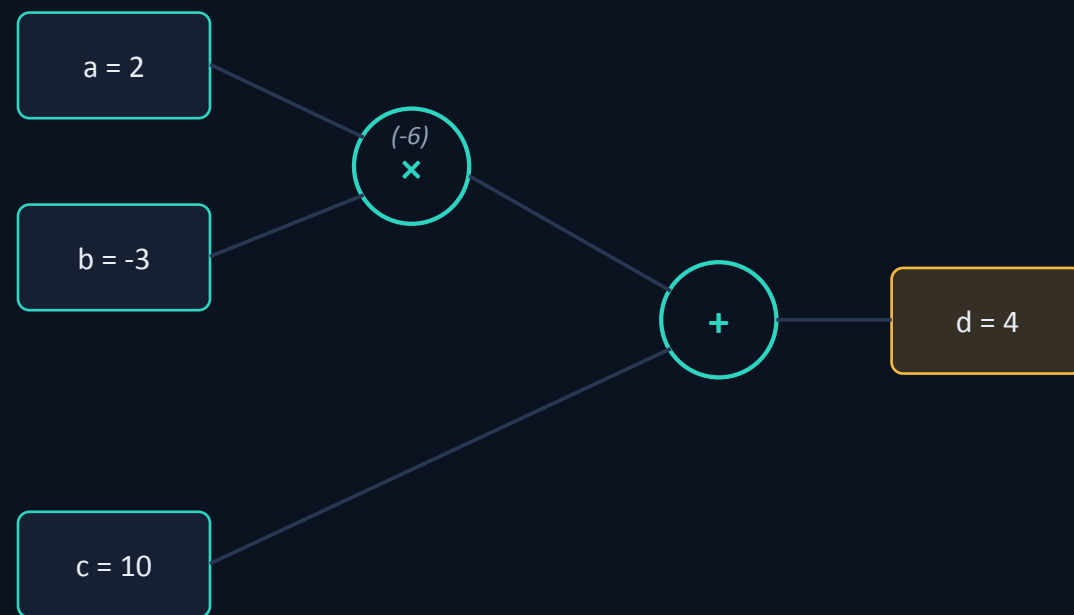
Grafo de cómputo

toda expresión = DAG

Nodos = valores intermedios; aristas = operaciones. Forward: evaluar en orden topológico. Backward: recorrer en orden inverso, acumulando $\partial L / \partial (\text{nodo})$ con la derivada local de cada operación.

ejemplo: $d = a \cdot b + c$

En el lab construimos exactamente esta estructura en código; un Transformer es un grafo gigante de estas mismas operaciones.



Laboratorio: motor de autograd

Construimos, no demostramos: la parte rigurosa de sistemas.

Objetivo: implementar un motor de diferenciación automática en modo reverso.

micrograd · ~100 líneas · Andrej Karpathy · licencia MIT

Método: notebook con celdas a completar — construir y verificar.

AL CIERRE, CADA UNO TIENE

- 1 Value con backward (nodo del grafo)
- 2 un MLP entrenable
- 3 verificación por gradient check



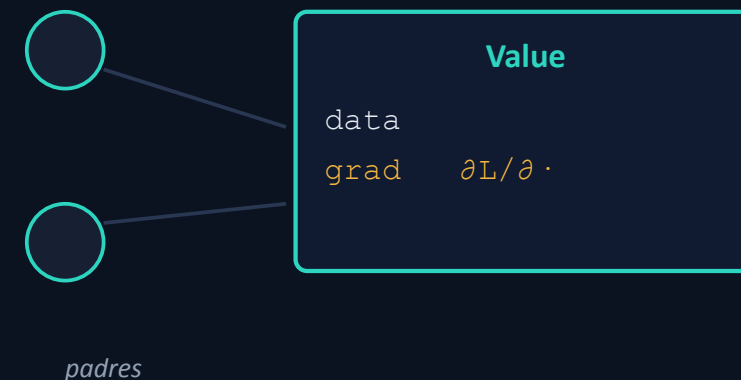
El nodo del grafo: Value

```
class Value:
    def __init__(self, data, _children=()):
        self.data = data
        self.grad = 0.0          #  $\partial L / \partial \text{self}$ 
        self._prev = set(_children)
        self._backward = lambda: None

    def __add__(self, other):
        out = Value(self.data + other.data,
                    (self, other))
        def _backward():          # regla de la cadena
            self.grad += 1.0 * out.grad
            other.grad += 1.0 * out.grad
        out._backward = _backward
        return out
```

POR QUÉ IMPORTA

Cada Value es un nodo del grafo: guarda su valor (data), su gradiente (grad) y una función que sabe propagar ese gradiente a sus padres.



Forward: operar y armar el grafo

```
def __mul__(self, other):
    out = Value(self.data * other.data,
                (self, other))
    def _backward():
        #  $\partial(x \cdot y) / \partial x = y$  ,  $\partial(x \cdot y) / \partial y = x$ 
        self.grad += other.data * out.grad
        other.grad += self.data * out.grad
    out._backward = _backward
    return out
```

```
# construir el grafo hacia adelante
a = Value(2.0); b = Value(-3.0); c = Value(10.0)
d = a*b + c      # d.data == 4.0 ; arma el DAG
```

POR QUÉ IMPORTA

Cada operación hace dos cosas a la vez: calcula el valor y agrega un nodo al grafo. La derivada local queda guardada en su `_backward`.



la derivada local vive en cada arista

Backward + gradient check

```
def backward(self):
    topo, seen = [], set()
    def build(v):
        # orden topológico
        if v not in seen:
            seen.add(v)
            for ch in v._prev: build(ch)
        topo.append(v)
    build(self)
    self.grad = 1.0
    #  $\partial L / \partial L = 1$ 
    for v in reversed(topo):
        # de la salida a la entrada
        v._backward()

# verificar: analítico  $\approx$  numérico
#  $\partial d / \partial a \approx (f(a+h) - f(a-h)) / 2h$ 
```

POR QUÉ IMPORTA

`backward()` ordena el grafo y lo recorre al revés, disparando cada `_backward`. Una función, todos los gradientes.



el gradiente fluye ←

gradient check

$\partial d / \partial a$ analítico -3.000 $\approx$ numérico
-2.999

Una neurona: muchas entradas, una salida

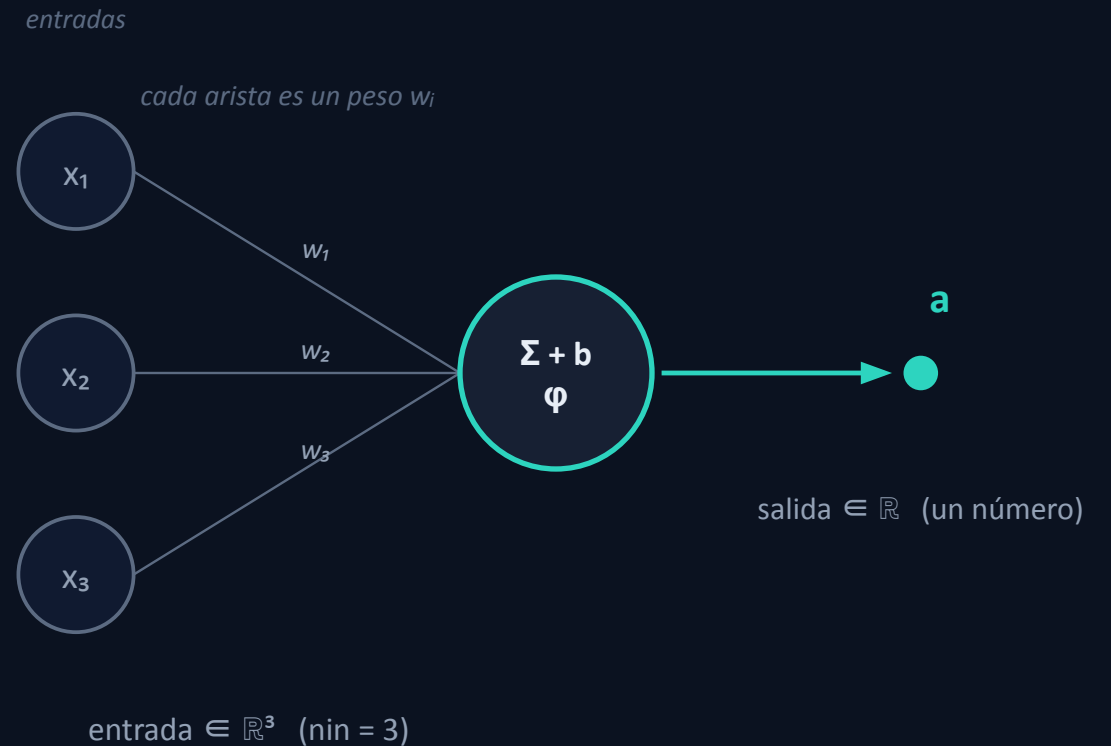
$$a = \varphi(\mathbf{w} \cdot \mathbf{x} + b)$$

Toma las n_{in} entradas, hace un producto interno con sus pesos, suma el sesgo y aplica la activación. Sale un solo número.

```
w = [Value(..)] # un peso por entrada
```

```
b = Value(0.0) # sesgo
```

n_{in} entradas $\rightarrow$ **1** salida



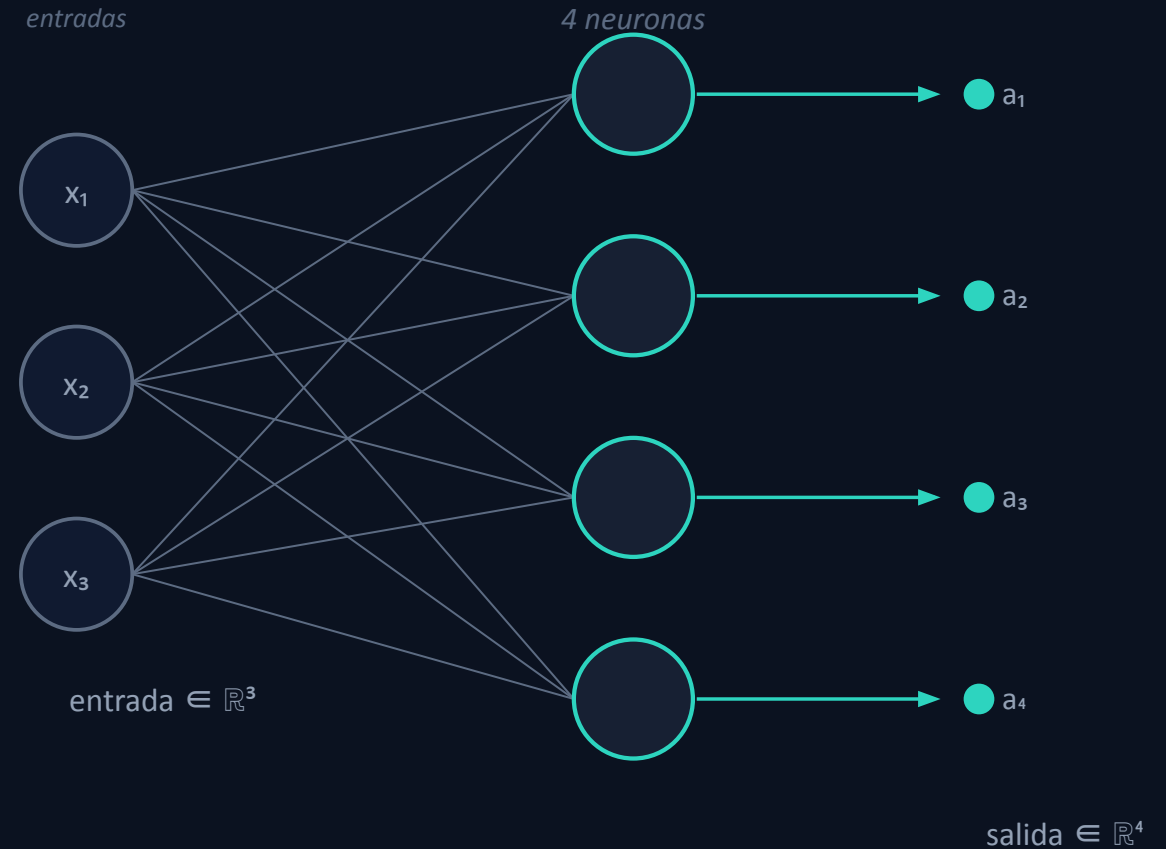
Una Capa (Layer): n_{in} entradas $\rightarrow$ n_{out} salidas

Una capa son n_{out} neuronas, todas sobre la MISMA entrada. Cada neurona da un número; juntos forman un vector de n_{out} .

```
neurons = [Neuron(nin) × nout]
```

Todas reciben x ; cada una tiene sus propios pesos y sesgo.

Layer(3, 4): $\mathbb{R}^3 \rightarrow \mathbb{R}^4$ ($n_{in}=3$, $n_{out}=4$)



El MLP: encadenar capas

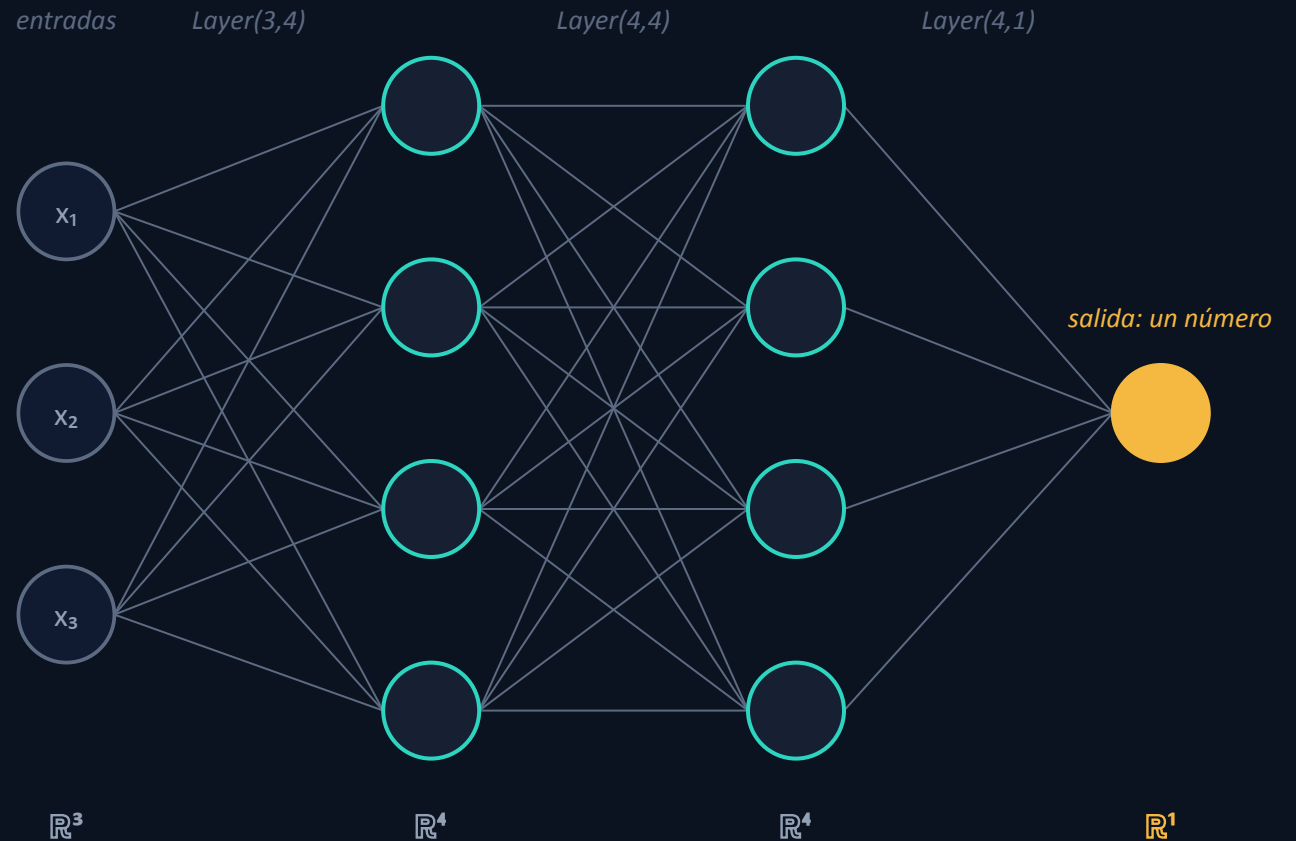
Un MLP encadena capas: la salida de una es la entrada de la siguiente. Las dimensiones tienen que calzar.

```
MLP(3, [4, 4, 1])
```

```
layers = [Layer(3,4),  
          Layer(4,4), Layer(4,1)]
```

$$\mathbb{R}^3 \rightarrow \mathbb{R}^4 \rightarrow \mathbb{R}^4 \rightarrow \mathbb{R}^1$$

última capa lineal (nonlin=False) → un número real



Los datos: un conjunto de vectores

xs no es un vector: es una LISTA de ejemplos. Cada ejemplo x es un vector de 3 números (las features). ys guarda el objetivo de cada ejemplo.

```
xs = [[ 2.0,  3.0, -1.0],  
      [ 3.0, -1.0,  0.5],  
      [ 0.5,  1.0,  1.0],  
      [ 1.0,  1.0, -1.0]]  
ys = [1.0, -1.0, -1.0, 1.0]
```

4 ejemplos · cada $x \in \mathbb{R}^3$ (3 features)

	x_1	x_2	x_3	objetivo
$x^{(1)}$	2.0	3.0	-1.0	→ 1.0
$x^{(2)}$	3.0	-1.0	0.5	→ -1.0
$x^{(3)}$	0.5	1.0	1.0	→ -1.0
$x^{(4)}$	1.0	1.0	-1.0	→ 1.0

cada fila = un ejemplo = un vector $(x_1, x_2, x_3) \in \mathbb{R}^3$

De neurona a red: el loop de entrenamiento

```

model = MLP(3, [4, 4, 1]) # 2 capas ocultas

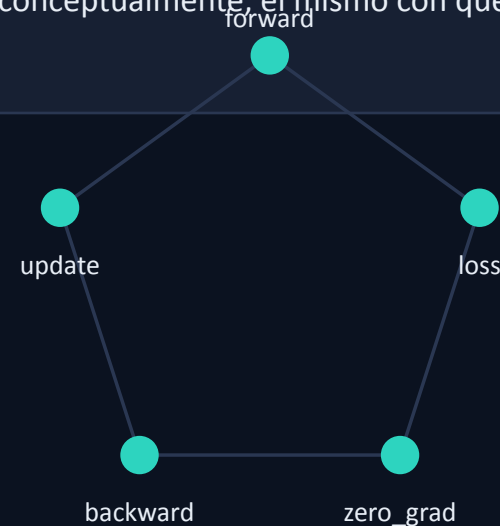
for step in range(epochs):
    ypred = [model(x) for x in xs]
    loss = sum((yp - yt)**2
               for yp, yt in zip(ypred, ys))

    for p in model.parameters():
        p.grad = 0.0 # zero_grad
    loss.backward() # backprop en el grafo

    for p in model.parameters():
        p.data -= lr * p.grad # descenso
  
```

POR QUÉ IMPORTA

Neuron → Layer → MLP: todo se apoya en Value. Este loop es, conceptualmente, el mismo con que se entrena un LLM.

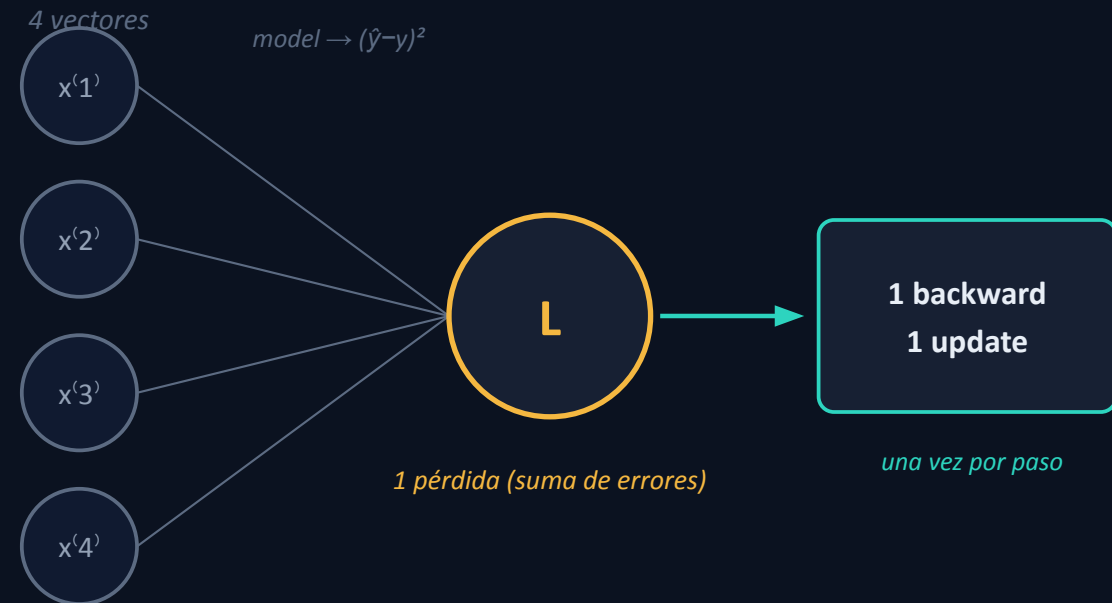


Un paso ve TODO el conjunto, no un vector

Confusión típica: pensar que cada vector hace su propio backward y update. En realidad, un paso predice los 4 vectores, suma sus errores en UNA pérdida, y hace UN backward y UN update.

```
ypred = [model(x) for x in xs] # 4 predicciones
loss = sum((yp-yt)**2 ...) # 1 pérdida (la suma)
loss.backward() # 1 backward
p.data -= lr*p.grad # 1 update
```

El for x in xs recorre los vectores DENTRO del forward. **Un solo loss, un solo backward, un solo update por paso.**



Un grafo, un backward: los 4 se juntan en la pérdida

Los 4 forwards no son grafos separados: comparten los mismos pesos θ y desembocan en una sola pérdida L .

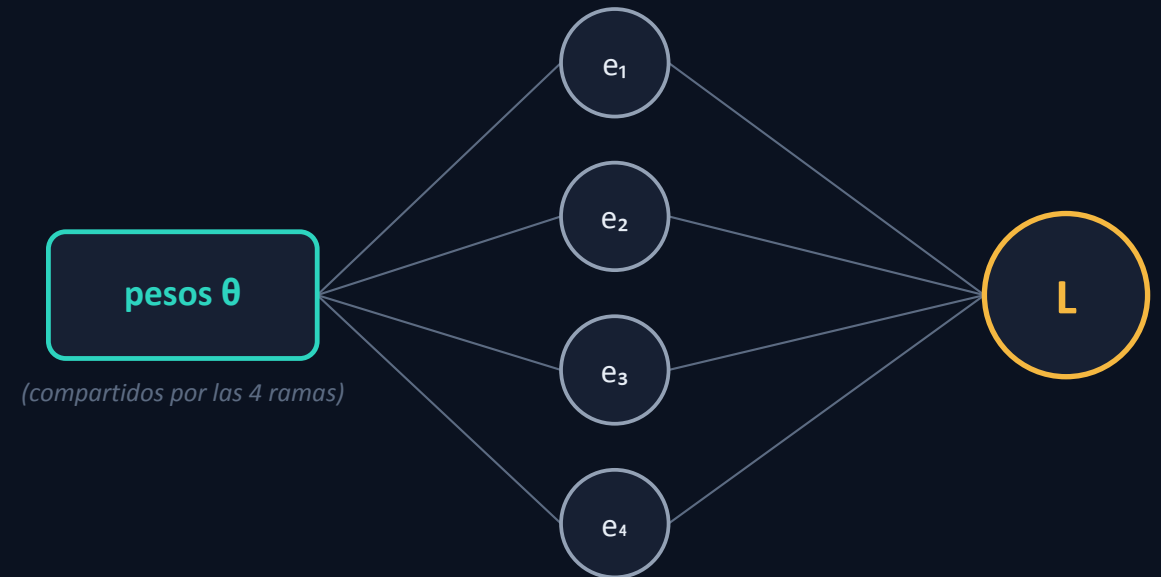
El backward hace UN solo barrido desde L . Como cada peso alimenta a las 4 ramas, le llegan 4 gradientes, y se acumulan:

$$\partial L / \partial w = \partial e_1 / \partial w + \partial e_2 / \partial w + \partial e_3 / \partial w + \partial e_4 / \partial w$$

el += suma las 4 contribuciones, solo

El backward no vuelve a mirar los vectores: **recorre el grafo (nodos y conexiones) que el forward ya dejó armado.**

FORWARD → 4 pasadas (una por vector) → 1 pérdida L



$$e_i = \text{error del ejemplo } i = (\hat{y}_i - y_i)^2$$

BACKWARD ← un solo barrido desde L ; en cada peso se acumulan (+=) las 4

Síntesis: la cadena, completa

Los mismos objetos, ahora entrenables.



implementado hoy, desde cero

DE ACÁ SALEN

Atención $\text{softmax}(QK^T/\sqrt{d})V$

Entrenamiento de LLMs

Agentes y multi-agente

Próxima clase. De estas piezas al Transformer: cómo la atención combina productos internos y softmax para procesar secuencias.

Trabajo formativo y lecturas

TRABAJO FORMATIVO

No entra en la nota; es la base conceptual de los proyectos.

- 1 Extendé el motor con tanh y exp.
- 2 Derivá a mano el gradiente local de cada una.
- 3 Verificá con gradient check.
- 4 Entrená un MLP en un dataset chico.
- 5 Reportá la curva de pérdida.

LECTURAS

Núcleo

- Goodfellow, Bengio & Courville, Deep Learning — cap. 2, 4 y 6.5
- CS231n (Stanford) — notas de Backprop y de Optimization
- Repositorio micrograd (A. Karpathy)

Opcional

- 3Blue1Brown, “Neural Networks” — cap. 1–4 (intuición visual)

Traé el notebook terminado: arrancamos la próxima sobre esta base.